

CompanyGraph – Was bleibt, wenn das Modell wechselt

Code, System One, System Two und ein Mensch: welches Modell Sie behalten und welches Sie ersetzen können.

BERUHT AUF concept Instance

Robert Blust · Software Engineer & Architect

Fünf Dinge heissen «Modell»

Zwei davon sind trainierte Modelle. Drei sind Dateien, die Sie behalten.

BERUHT AUF

concept Core

concept Instance

concept Type

SYSTEM TWO

Opus, Fable, Gemini Flash schreibt und zieht Schlüsse

SYSTEM ONE

Jev von TypeSafe beantwortet typisierte Fragen

META-MODELL

CompanyGraph core das Vokabular und seine Regeln

MENTAL MODEL

die Instanz einer Firma ihre Fakten, in diesem Vokabular

GRAPH

was daraus abgeleitet wird eine Website, ein Server, ein Chat

Vier Ebenen, eine Regel

Jeder Entscheid geht an die tiefste Ebene, die ihn treffen kann.

BERUHT AUF

decision Agents write the model, and a person approves every change to it

4 **Mensch**
gibt frei

3 **System Two**
schreibt und liest · Opus, Fable, Gemini Flash

2 **System One**
beantwortet typisierte Fragen · Jev

1 **Code**
entscheidet, was Code entscheiden kann · die Prüfung

Fest: das Meta-Modell

Das Vokabular und die Regeln, nach denen jede Seite geschrieben wird – in Markdown und in einem Release, das eine Instanz pinnt.

BERUHT AUF `concept Schema`

`decision The schema written as prose is the only schema`

`decision A release is its tag, and nothing we make is published to a registry`

meta/core/feature-schema.md

Writing rules

- The tagline says what someone can now do, in their words, and never names a screen, a service or a vendor.
- The H1 names the capability, not the vendor that supplies it.
- The page says nothing about a release, a ticket or a delivery date: those move, and a feature outlives all three.

Fest: das Mental Model

Die Fakten einer Firma als Markdown in git, geprüft an dem Release, das die Instanz übernommen hat. Das Mental Model gehört keinem Werkzeug und keinem Modell.

BERUHT AUF `concept Instance`

`strategic-objective` A company's model belongs to no tool, and any graph database can hold it

`decision` Every entity carries an id that outlives its name

model/features/...writing-rules.md

`id:` 01a10770-fd86-7891-a8a0-5ea8e270b7f2

`products:` CompanyGraph Core

`concepts:` Agent pass, Schema, Rule, Instance

A decision model asks each page its schema's writing rules

> A company learns which of its pages most likely break a writing rule before anyone reads them...

4 Mensch

3 System Two

2 System One

1 Code

Code: die Prüfung

Verweise, Schemas, IDs. Bei jedem Lauf dieselbe Antwort, ohne dass ein Modell gefragt wird.

BERUHT AUF `concept Check` `feature Checks an instance runs`

```
decision A reference resolves by its declared type, never by name alone
```

...

```
$ companygraph check
✓ model/ against meta/core at core <release>: the mechanical checks pass
  not checked: every `## Writing rules` in every schema – that is the
  agent pass, R0
✓ Markdown files in the one form
```

System One: der Judge

4 Mensch

3 System Two

2 System One

1 Code

Jede Schreibregel wird zu einer typisierten Frage. Jev antwortet mit einer Wahl und einer Wahrscheinlichkeit. Der Owner sagt Ja zu genau dem, was den Rechner verlässt.

Der Judge berät und ändert nichts.

BERUHT AUF

feature A decision model asks each page its schema's writing rules

question Does our data leave our hands?

```
$ companygraph judge --consent <digest>
judge: advisory – it gates nothing and is no pass

model/features/checkout.md
! 0.31 r1 The tagline says what someone can now do, in their words,
and never names a screen, a service or a vendor.
```

System Two: schreibt und liest

4 Mensch

3 System Two

2 System One

1 Code

Skills liefern Entwürfe. Der Skill validate liest zuerst die Markierungen des Judges, dann jede Seite. Der Server und der Chat antworten und nennen, worauf jede Antwort beruht.

BERUHT AUF `concept Agent pass`

```
feature An agent does the work no script can, by the skills an
instance ships
```

```
decision The server only reads, and answers at one named commit
```

companygraph-validate

Mechanical checks passed

Writing rules

```
model/features/checkout.md breaks feature r1: "The tagline says what
someone can now do ..."
```

Not checked: nothing

4 Mensch

3 System Two

2 System One

1 Code

Ein Mensch gibt frei

Ein Durchgang, der nichts findet, gibt nichts frei. Gemergt wird erst, wenn der Owner es sagt.

BERUHT AUF `rule The Owner's word merges`

`strategic-objective` Every gate an agent's work passes is held by a person

`control` A reviewer reads the output, not the report

pull request

companygraph / companygraph ✓

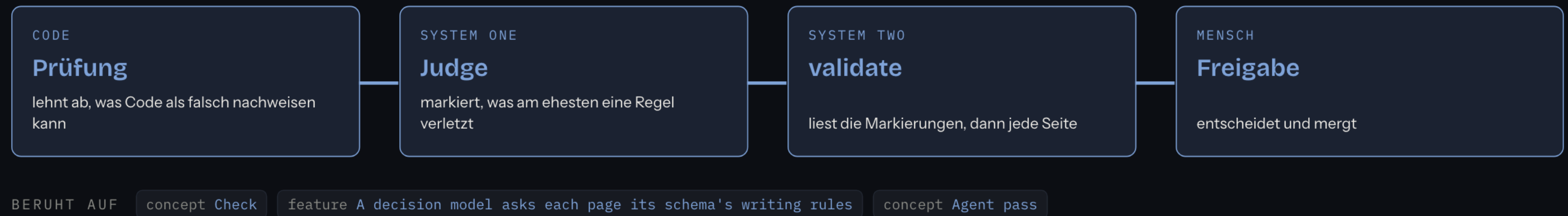
conventions / conventions ✓

merge

on the Owner's word

Der Ablauf

Jeder Schritt tut, was er gut kann, und sagt, was er dem nächsten überlässt.



Fest und austauschbar

Was bleibt, ist, was Sie geschrieben haben. Was wechselt, ist, was es liest – und bei jedem Teil steht, ob es heute schon austauschbar ist oder dem Design nach.

BERUHT AUF

decision Claude is the first agent an instance supports, and not the only one

strategic-objective A company keeps its model with whichever agent it chooses

question Does it only work with Claude, or can we use ChatGPT or Gemini?

FEST

das Meta-Modell

Ihr Mental Model

die Prüfung

Ihre Regeln und Entscheide

die Freigabe durch einen Menschen

AUSTAUSCHBAR

der Agent, der über MCP liest

heute

der Editor

heute

das System-One-Modell

dem Design nach

das System-Two-Modell, das schreibt

dem Design nach

die Graphdatenbank

dem Design nach

Messen, bevor Sie wechseln

Jedes Modell ist gepinnt. Ein neues wird an derselben Firma gemessen, bevor es an seine Stelle tritt.

BERUHT AUF `kpi Grounded Answer Rate`

`question Can I trust what this chat says?`

`rule A check counts only when its output was seen`

`bin/judges/typesafe.mjs`

```
// The one place the tooling names a judge ... so a second judge is a second
// file beside this one and the questions do not change. The model is pinned
// rather than `jev-latest`, because the band the report flags by is measured
// against one model, and a newer one is measured again before it is named here.
```

Sechs Praktiken

- 01 Legen Sie Wissen dort ab, wo es keinem Modell gehört.
- 02 Lassen Sie Code entscheiden, was Code entscheiden kann.
- 03 Stellen Sie typisierte Fragen, bevor Sie freien Text verlangen.
- 04 Ein Modell schlägt vor, ein Mensch entscheidet.
- 05 Pinnen Sie jedes Modell, und messen Sie, bevor Sie den Pin verschieben.
- 06 Fragen Sie, bevor Daten den Rechner verlassen.

BERUHT AUF `value Run on what we publish`

Was bleibt

Von dem Modell, das Sie am häufigsten wechseln werden, sollten Sie am wenigsten abhängen.

Dann ist das nächste Modell ein Pin, den Sie verschieben, und kein Projekt, das Sie beginnen.

BERUHT AUF

decision Agents write the model, and a person approves every change to it

4 **Mensch**
gibt frei

3 **System Two**
schreibt und liest · Opus, Fable, Gemini Flash

2 **System One**
beantwortet typisierte Fragen · Jev

1 **Code**
entscheidet, was Code entscheiden kann · die Prüfung

Quellen und Dank

ANGEREGT DURCH

TypeSafe, das Jev, sein Entscheidungsmodell, als System-One-Modell bezeichnet – als Gegenstück zu den System-Two-Modellen, die Text erzeugen.

SYSTEM ONE UND SYSTEM TWO

Daniel Kahneman, *Thinking, Fast and Slow*, 2011; zuerst benannt von Keith Stanovich und Richard West, 2000

JEV UND DIE API, DIE DER JUDGE AUFRUFT

typesafe.ai · docs.typesafe.ai

DIE MODELLE DIESES VORTRAGS

companygraph/meta-model · companygraph/mental-model